UNIVERSIDADE DE FORTALEZA

Fundação Edson Queiroz

Curso de Engenharia da Computação

RELATÓRIO DE ATIVIDADE PRÁTICA: PRÁTICA 4

Conversão Analógico-Digital (ADC) com Interrupção e Leitura em Volts na Placa NUCLEO-STM32F446RE

Disciplina: Sistema de Tempo Real (STR)

Professor: Imbiriba

Aluno: Rafael Marins

Fortaleza, Outubro de 2026

1. Introdução

Os sinais do mundo físico, como temperatura, luminosidade ou a posição de um potenciômetro, são grandezas analógicas, enquanto um microcontrolador só processa números. O conversor analógico-digital (ADC) faz essa ponte: mede a tensão presente em um pino e a transforma em um valor inteiro que o programa pode comparar, calcular ou enviar a outro dispositivo.

Este relatório documenta a Prática 4 da disciplina, que explora o ADC1 do microcontrolador STM32F446RE da placa NUCLEO-F446RE por acesso direto aos registradores (sem uso da biblioteca HAL), seguindo a mesma metodologia das Práticas 1 a 3. A tensão do pino PA1 (canal 1 do ADC1) é lida e o resultado é enviado pela porta serial (USART2) a um terminal no computador. A prática evolui em cinco etapas: simples conversão por polling (Parte 1), conversão contínua por polling (Parte 2), conversão contínua com fim de conversão por interrupção (Parte 3), conversão simples disparada pelo botão da placa (Tarefa 1) e conversão do valor lido para Volts, enviado como número de ponto flutuante (Tarefa 2).

Além da implementação, o relatório registra os testes em bancada de cada etapa e as observações feitas durante a montagem.

2. Objetivos

Objetivo geral: compreender e configurar o ADC do STM32F446 em nível de registradores, nos modos de conversão simples e contínua, com detecção do fim da conversão por polling e por interrupção, e transmitir os resultados pela serial.

Objetivos específicos:

- Configurar o pino PA1 como entrada analógica e o ADC1 para converter o canal 1;
- Realizar conversões simples (Parte 1) e contínuas (Parte 2), esperando o término pela flag EOC (End Of Conversion);
- Habilitar a interrupção de fim de conversão (bit EOCIE) e tratá-la em ADC_IRQHandler (Parte 3);
- Disparar uma conversão simples ao pressionar o botão B1 (PC13) e enviar o resultado pela serial, com o fim da conversão detectado por interrupção (Tarefa 1);
- Converter o valor digital de 12 bits em Volts e transmiti-lo com printf em ponto flutuante (Tarefa 2);
- Validar cada etapa na placa física, observando os resultados em um terminal serial.

3. Revisão Bibliográfica

O conversor analógico-digital por aproximação sucessiva compara a tensão de entrada, mantida constante por um circuito de amostragem e retenção (sample & hold), com a saída de um conversor digital-analógico (DAC) interno. Um bloco de aproximação ajusta o valor do DAC bit a bit, do mais significativo ao menos significativo, até que a saída do DAC se iguale à entrada. O tempo de conversão é, portanto, proporcional ao número de bits [5]. O ADC dos microcontroladores STM32F4 possui resolução de 12 bits (4096 níveis) e até 16 canais externos multiplexados, e pode operar nos modos de conversão simples, contínua, varredura (scan) e descontínua [1].

O resultado da conversão fica no registrador de dados ADC_DR, alinhado à direita (valor nos bits 11:0) ou à esquerda, e a flag EOC do registrador de status ADC_SR indica que há um resultado disponível; a leitura de ADC_DR apaga essa flag automaticamente [1]. O término da conversão pode ser detectado por varredura (polling) da flag EOC ou por interrupção, habilitando o bit EOCIE do registrador ADC_CR1 e a requisição ADC_IRQn no controlador de interrupções do núcleo (NVIC). Nesse caso, a rotina de tratamento é a função ADC_IRQHandler, declarada como símbolo fraco no arquivo de inicialização e sobrescrita pelo usuário [1][2].

Como em toda a família STM32F4, cada periférico fica ligado a um barramento (AHB1, APB1 ou APB2) e precisa ter o seu clock habilitado nos registradores do módulo RCC antes do uso; o GPIOA fica no AHB1, o ADC1 no APB2 e a USART2 no APB1 [1]. O pino que será lido pelo ADC deve ainda ser colocado em modo analógico no registrador GPIOx_MODER, o que desliga o buffer digital do pino [1].

A conversão do valor digital para tensão é linear. Para um ADC de 12 bits com tensão de referência de 3,3 V, o passo de quantização (LSB) é de 3,3/4095, aproximadamente 0,806 mV [1]. Finalmente, as operações de ponto flutuante executadas em hardware dependem da unidade FPU do núcleo Cortex-M4, cujo acesso é habilitado pelo registrador CPACR [2][3], e o printf com %f exige, na biblioteca newlib-nano, a opção de linkedição -u _printf_float.

4. Metodologia

4.1 Ambiente de desenvolvimento

O desenvolvimento foi realizado no STM32CubeIDE 2.2.0, em um projeto configurado para o STM32F446RETx (placa NUCLEO-F446RE), com acesso direto aos registradores. O código está organizado em módulos: uart.c/uart.h (USART2, reaproveitado da Prática 3), adc.c/adc.h (ADC1) e main.c, que orquestra o experimento. Uma diretiva #define PRATICA_PARTE, de 1 a 5, seleciona qual etapa é compilada: 1, 2 e 3 correspondem às Partes 1, 2 e 3, e 4 e 5 correspondem às Tarefas 1 e 2.

Embora o roteiro e o Manual de Referência RM0383 tratem do STM32F411, os registradores de RCC, GPIO, ADC e USART utilizados são idênticos aos do STM32F446, por pertencerem ao mesmo IP da família STM32F4. A placa efetivamente usada é a NUCLEO-F446RE e o código funcionou sem adaptações.

4.2 Configuração do ADC

A função pa1_adc_init() prepara o ADC1 para converter o canal 1, na seguinte ordem (a ordem é importante, pois a escrita em registradores de um periférico sem clock é ignorada):

- Habilitação do clock do GPIOA (RCC_AHB1ENR) e configuração de PA1 em modo analógico (GPIOA_MODER, bits 3:2 = 11);
- Habilitação do clock do ADC1 (RCC_APB2ENR, bit ADC1EN);
- Seleção do canal 1 como primeira conversão da sequência (ADC1_SQR3 = 1) e comprimento da sequência igual a uma conversão (ADC1_SQR1 = 0, pois o campo L guarda o número de conversões menos um);
- Habilitação do módulo (ADC1_CR2, bit ADON). Essa função não inicia nenhuma conversão.

4.3 Partes 1 e 2: polling

Na Parte 1, a cada iteração do laço principal a função start_convertion() dispara uma conversão (bit SWSTART de ADC1_CR2), após garantir que o modo contínuo está desligado (CONT = 0). Em seguida, adc_read() aguarda a flag EOC em ADC1_SR e devolve o conteúdo de ADC1_DR, que é impresso na serial com printf. Na Parte 2, a função start_continuous_convertion() ativa o bit CONT e dispara a conversão uma única vez; a partir daí o ADC reinicia sozinho a cada fim de conversão, e o laço apenas chama adc_read() para obter o valor mais recente. Um laço de atraso entre as impressões evita inundar o terminal.

4.4 Parte 3: interrupção

A função pa1_adc_interrupt_init() executa a inicialização anterior e, em seguida, habilita o bit EOCIE de ADC1_CR1 e a interrupção ADC_IRQn no NVIC (NVIC_EnableIRQ). A conversão contínua é iniciada uma vez e o laço principal fica vazio: a cada fim de conversão a CPU executa ADC_IRQHandler(), que verifica a flag EOC, a limpa e chama adc_callback(), responsável por ler ADC1_DR e imprimir o valor, conforme o roteiro da prática.

4.5 Tarefa 1: botão e interrupção

O botão B1 do usuário está ligado ao pino PC13 e é ativo em nível baixo (0 = pressionado), por isso o resultado da leitura de GPIOC_IDR é invertido na função button_pressed(). O laço principal detecta a borda de pressionamento (antes solto, agora pressionado) e somente nesse instante chama start_convertion(), de modo que cada clique gera exatamente uma conversão simples, mesmo que o botão permaneça apertado. Um atraso curto funciona como debounce.

O fim da conversão é detectado por interrupção: ADC_IRQHandler() lê ADC1_DR (o que também limpa a flag EOC), guarda o valor em adc_raw e levanta a variável adc_done. As duas variáveis são declaradas volatile, pois são alteradas na rotina de interrupção e lidas no laço principal. O printf é executado no laço principal, e não na interrupção, para que a rotina de tratamento permaneça curta, já que a transmissão serial é lenta.

4.6 Tarefa 2: conversão para Volts

O valor digital é convertido para tensão segundo a Equação 1, considerando a referência de 3,3 V da placa, e enviado com printf("%.3f V"):

$$V = (valor_digital \times 3{,}3) / 4095 \qquad (Equação\ 1)$$

Para o uso de float foram necessárias duas providências. Primeiro, a opção de linkedição -u _printf_float (em C/C++ Build → Settings, equivalente a marcar "Use float with printf from newlib-nano"), pois a newlib-nano não inclui por padrão o suporte a float no printf. Segundo, como o projeto usa hard float (-mfloat-abi=hard, fpv4-sp-d16), a FPU é habilitada no início do main com SCB->CPACR |= (0xF << 20); sem isso, a primeira instrução de ponto flutuante provocaria uma falha de hardware (HardFault).

4.7 Validação física

A USART2 (PA2/TX e PA3/RX) é a interface que a placa NUCLEO conecta, de fábrica, ao ST-LINK integrado, que a disponibiliza ao computador como uma porta serial virtual (/dev/ttyACM0). Por isso, diferentemente de uma ligação em outra USART, não foi necessário o adaptador USB-TTL externo: bastou o cabo USB da própria placa. Os dados foram visualizados no terminal minicom (Linux), configurado para 115200 bps, 8N1 e sem controle de fluxo.

A entrada analógica PA1 corresponde ao pino A1 do conector CN8 (Arduino). Como não havia potenciômetro disponível, a tensão aplicada foi controlada manualmente por um jumper ligado ao pino de GND ou ao de 3V3 do conector de energia CN6. Nos testes das Partes 1, 2 e 3, o jumper foi movido entre GND, 3V3 e a condição sem conexão enquanto o programa imprimia continuamente.

5. Resultados e Discussão

5.1 Parte 1: simples conversão por polling

Na Parte 1 o programa imprime continuamente o valor lido (Figura 1). Com o jumper em 3V3 as leituras ficaram em 4093 a 4095, o valor máximo da escala de 12 bits. Ao retirar o jumper, a entrada ficou sem referência definida e os valores passaram a variar de forma instável (de 0 a cerca de 1000), como esperado para uma entrada analógica flutuante que capta ruído e carga residual. Com o jumper em GND as leituras ficaram em 0 e 1.

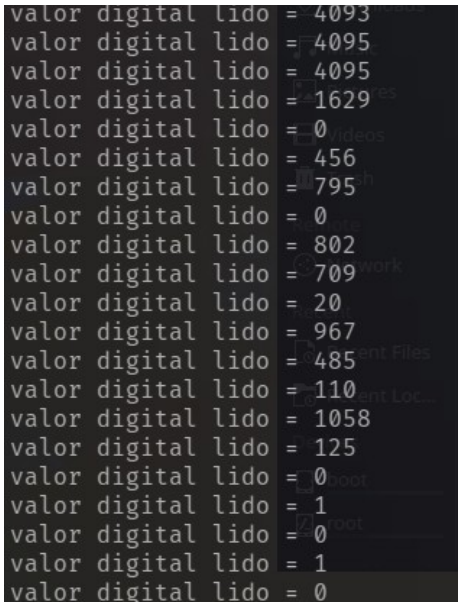


Figura 1 — Parte 1 (polling): valores lidos com o jumper em 3V3 (≈ 4095), com a entrada flutuante (valores instáveis) e em GND (0 ou 1).

5.2 Parte 2: conversão contínua

A Parte 2 mostra o mesmo comportamento com o ADC convertendo continuamente (Figura 2). Com a entrada em GND, as leituras ficaram em 0; com a entrada sem referência definida, estabilizaram em torno de 1000 a 1070; e com o jumper em 3V3 chegaram ao fundo de escala, 4095. A variação do valor

em tempo real, sem que o programa reinicie a conversão, confirma o modo contínuo: o ADC reinicia sozinho após cada conversão e o laço principal apenas lê o resultado mais recente.

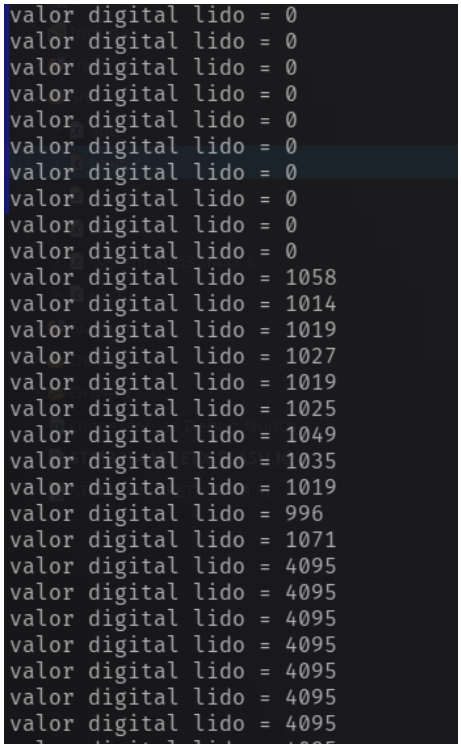


Figura 2 — Parte 2 (contínuo): leituras acompanham a tensão em PA1 em tempo real (0, valores intermediários e 4095).

5.3 Parte 3: conversão com interrupção

Na Parte 3 as leituras são obtidas e impressas dentro da rotina de interrupção do ADC (Figura 3). O resultado é o mesmo da Parte 2, com valores em 4095, valores intermediários e 0 conforme o jumper era movimentado, o que confirma que a requisição ADC_IRQn está sendo gerada, habilitada no NVIC e atendida por ADC_IRQHandler.

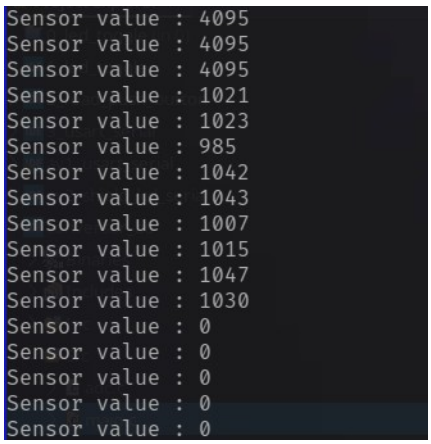


Figura 3 — Parte 3 (interrupção): saída "Sensor value" impressa a cada fim de conversão, variando de 4095 a 0 conforme a tensão em PA1.

5.4 Tarefa 1: conversão ao pressionar o botão

Na Tarefa 1 nenhuma linha é impressa até que o botão azul B1 seja pressionado; cada clique gera exatamente uma conversão e uma linha na serial (Figura 4). Com a entrada em GND, os cliques resultaram em 0 e 1; ao ligar a entrada em 3V3, os cliques seguintes resultaram em 4095. Isso confirma o disparo por software com o botão, a detecção do fim da conversão por interrupção e o envio do resultado pela serial.

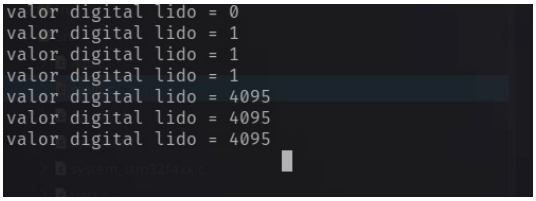


Figura 4 — Tarefa 1: uma linha por clique do botão B1, com PA1 em GND (0 e 1) e depois em 3V3 (4095).

5.5 Tarefa 2: valor em Volts

Na Tarefa 2, o valor passou a ser enviado em Volts, com três casas decimais. A Figura 5 mostra os dois extremos: com PA1 em GND foi lido 0,000 V e, com PA1 em 3V3, 3,300 V, coincidindo com os valores esperados. Isso valida a equação de conversão, o canal selecionado e o suporte a ponto flutuante (FPU habilitada e opção -u _printf_float efetiva). Com o pino sem ligação, uma das leituras foi 0,157 V, valor sem significado, característico de uma entrada flutuante.

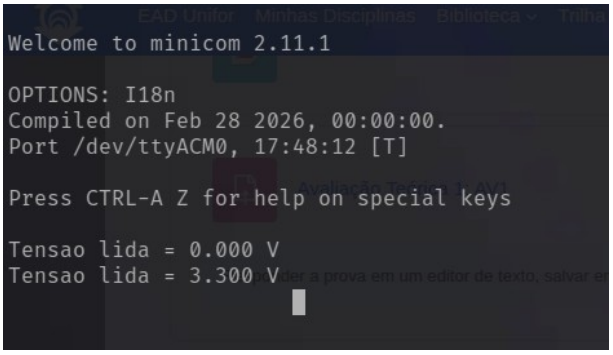


Figura 5 — Tarefa 2: saída no minicom (115200 bps) com PA1 em GND (0,000 V) e em 3V3 (3,300 V).

Um teste inicial dessa etapa retornou 3,3 V com o jumper que deveria estar em GND. A causa foi uma ligação equivocada a um pino de 3,3 V entre os pinos de alimentação vizinhos do conector Morpho (CN10). Ao ligar o jumper a um pino GND do conector CN6, a leitura passou a 0 V, o que mostrou que o código estava correto e que o erro era de fiação. O episódio ressalta a importância de conferir os rótulos dos pinos de alimentação, já que a entrada do ADC não tolera tensões acima de 3,3 V.

5.6 Observações

Os testes validam a escala e o funcionamento de cada modo nos extremos (0 e 3,3 V), mas não a linearidade da conversão nos valores intermediários, pois os valores intermediários observados vieram de uma entrada sem referência definida e não de uma tensão conhecida. Essa validação exige um potenciômetro, como indica o roteiro, e fica como trabalho futuro. O tempo de uma conversão simples é

de aproximadamente 15 ciclos do clock do ADC (estimativa pelo clock padrão, não medida), muito menor que o intervalo entre leituras impresso na serial.

6. Conclusão

A atividade permitiu configurar o ADC1 do STM32F446 por acesso direto aos registradores e operá-lo nos modos de conversão simples e contínua, com o fim da conversão detectado por polling e por interrupção. As cinco etapas (Partes 1, 2 e 3 e Tarefas 1 e 2) foram executadas na placa e os resultados no terminal serial foram coerentes com a tensão aplicada: 0 V resultou em 0 e 3,3 V resultou em 4095, ou 3,300 V na conversão para Volts.

A comparação entre polling e interrupção mostrou o ganho da segunda: em vez de a CPU ficar presa aguardando a flag EOC, ela executa o laço principal e só atende o ADC quando há resultado, desde que a rotina de interrupção seja curta, o que justificou mover o printf para o laço principal na Tarefa 1. O uso de float no printf exigiu a opção -u _printf_float no linker e a habilitação da FPU, e a falta de qualquer uma das duas compromete a saída ou trava o sistema. A serial virtual do ST-LINK dispensou o adaptador USB-TTL externo.

Como trabalho futuro, propõe-se validar a linearidade do conversor com um potenciômetro linear de aproximadamente 10 kΩ ligado entre 3V3 e GND, com o cursor em PA1, comparando a tensão calculada com a medida por um multímetro.

Referências

[1] STMICROELECTRONICS. RM0383 — Reference manual: STM32F411xC/E advanced Arm® -based 32-bit MCUs. Rev. 3, nov. 2018. (Os registradores de RCC, GPIO, ADC e USART são os mesmos do STM32F446.)

[2] STMICROELECTRONICS. PM0214 — STM32F3 Series, STM32F4 Series, STM32L4 Series and STM32L4+ Series Cortex® -M4 programming manual.

[3] STMICROELECTRONICS. Datasheet: STM32F446xC/E — Arm® Cortex® -M4 32b MCU+FPU.

[4] STMICROELECTRONICS. UM1724 — User manual: STM32 Nucleo-64 boards (MB1136). Rev. 5, jun. 2014.

[5] IMBIRIBA. Prática 4 — Conversor analógico digital. Disciplina Sistema de Tempo Real (STR). Universidade de Fortaleza (UNIFOR), 2026.